COURSE BRIEF · AFTER THE LLM INTERNALS SESSIONS

Architecture Study, LoRA and SLM Fine-Tuning

This brief describes the three pieces of work for this module and the public writing that goes with them.

WHY THIS BRIEF EXISTS

In the last two sessions we followed one token through Qwen: the tokenizer, the embedding lookup, normalization, attention, the feed-forward network and the output head. We also saw how a LoRA adapter adds a small set of trainable weights to a frozen model.

This brief asks you to use that knowledge three times. You read other model papers and explain what changed and why. You explain LoRA visually in a video. Then you fine-tune a small model so that it learns a behavior from data that you made.

Use ChatGPT, Claude, YouTube and blog posts to learn. The work that you submit must show your own understanding. We expect you to find most of the answers yourself.

HOW TO READ THIS BRIEF

Must means required.
Should means expected. If you skip it, say why.
Can means optional.

Section references look like [A.3].

PART A · RESEARCH

The White Paper

Compare **Qwen**, **Llama** and **Mistral** with the abstraction lens from class.

YOU SUBMIT

- A white paper in LaTeX, 10 to 15 pages.
- A comparison table of the main blocks.
- At least four figures that you drew.
- A reading log.

GOOD WORK SHOWS

You can find the block that changed in a new paper, and the problem behind the change.

PART B · EXPLAIN

The LoRA Video

Explain LoRA visually: the matrices A and B, the rank and alpha.

YOU SUBMIT

- A video of 10 to 20 minutes, made with Manim.
- A short report, 4 to 6 pages.
- The animation source code.

GOOD WORK SHOWS

A viewer who knows the token path from class understands why LoRA works, without the math.

PART C · BUILD

The SLM Project

Fine-tune a small Qwen model on **Tinker** with your own dataset.

YOU SUBMIT

- A public repository with a step-by-step commit history.
- Your dataset and review SOP.
- Baseline and frontier scores.
- One LoRA run and a scorecard.

GOOD WORK SHOWS

The fine-tuned model does the behavior more consistently than the base model with a prompt.

PART D

200

Build in public

Post 150 to 200 pieces of content on X and LinkedIn while you work on Parts A, B and C. There is no deadline. Each post covers one thing you learned, built, measured or got wrong. Write in your own words. You can use AI tools to draw figures and fix grammar, but the thinking must be yours.

PART A · RESEARCH

The White Paper

Study Qwen, Llama and Mistral. Explain what each design choice does, why the authors made it, and what it costs.

TEAMS You can do this part alone or in a team. To find teammates, post on Skool. In your submission, list what each person did.

A.1

Goal

The goal is to understand why each architecture looks the way it does. You do not need to derive equations. You must explain what each block does, why the authors chose it, and what it costs. If a reader finishes your paper and can reason about a new model in the same family, the paper did its job.

Write as a generalist. A good paper connects many models through the same few ideas. It does not list every detail of one model.

A.2

Scope

- You **must** cover Qwen, Llama and Mistral. Pick one version of each (for example Qwen2.5, Llama 3 and Mistral 7B) and say which one you use.
- You **can** add other dense decoder-only models if they help your argument, for example Gemma or Phi.
- You **should** leave out mixture-of-experts models such as DeepSeek. We cover them in a later session.
- You **should** leave out inference topics, such as the KV cache and how a model generates text. We cover inference in a later session.

A.3

The abstraction lens

In class we split a model into blocks. Each block has one job. Use the same blocks to compare the three models. Fill in this table for your paper. Each cell should say what the authors chose and, in one line, why. The table is a minimum. If you find another useful lens, add a row for it, for example the activation function or whether the input and output embeddings share weights.

BLOCK	THE QUESTION IT ANSWERS	QWEN	LLAMA	MISTRAL
Tokenizer and vocabulary	How is text split, and how many tokens can the model know?	?	?	?
Embedding width	How many numbers describe one token?	?	?	?
Normalization	How are values kept on the same scale, and where?	?	?	?
Positional encoding	How does the model know word order?	?	?	?
Attention	How does a token decide which earlier tokens matter?	?	?	?
Feed-forward network	What does the token do with the context it gathered?	?	?	?
Depth	How many times is the block repeated?	?	?	?
Output head	How does the last vector become a next-token prediction?	?	?	?
Context length	How many tokens can the model read at once?	?	?	?
Your own lens	Add as many rows as you need.			

A.4

Topics you must explain well

These topics carry the most weight in the rubric [E]. Explain each one so that a classmate who missed the sessions can follow it. Keep the math light. Say what the idea is, why it is needed, and why it is a sensible choice.

1. Tokenizers. Explain how a tokenizer turns text into token IDs, and what happens to a word that is not in the vocabulary. Compare the main types: word-level, character-level and subword tokenizers. For subword tokenizers, explain byte-pair encoding (BPE), byte-level BPE, WordPiece and Unigram (SentencePiece). Then say which type each of the three models uses, and how the vocabulary size changes cost and quality.

2. Positional encoding. Attention alone does not know the order of tokens. "The dog sat on the mat" and "the mat sat on the dog" look the same to it. Explain the main ways to add order: learned positions, sinusoidal encodings and rotary encodings (RoPE). Explain why most current models use RoPE, and what happens when the input is longer than the length the model was trained on.

3. Attention heads. In class, one 64-number vector was split into two heads of 32 numbers each. Explain why models do this. Explain how the number of heads relates to the model width, and what different heads tend to learn.

4. Grouped-query attention (GQA). Compare three designs: every query head has its own keys and values (multi-head attention), all query heads share one set (multi-query attention), and groups of query heads share a set (GQA). Explain why a model would let query heads share keys and values, what it can cost in quality, and which of the three models use it. Do not cover the inference side of GQA. We cover it in the inference session.

5. Hyperparameters and their trade-offs. For each hyperparameter, say what it controls, what you gain when you make it larger, and what it costs. Use the table below as a starting point. The first row is done for you from class.

HYPERPARAMETER	WHAT IT CONTROLS	IF YOU MAKE IT LARGER	WHAT IT COSTS
FFN expansion	How far the token vector is widened before it is compressed back (64 to 192 in class, so 3×).	More room to combine features and decide what to do with the context.	More parameters and more compute in every block.
Model width			
Number of heads			
Key–value heads			
Number of blocks			
Vocabulary size			
Context length			

6. Topics we did not cover in class. Pick at least two of these and write a short section on each. One or two paragraphs is enough.

- Sliding-window attention.
- The SwiGLU activation and gated feed-forward networks.
- Normalization before or after the block (pre-norm and post-norm).
- Shared input and output embeddings (weight tying).
- Ways to extend the context length of a trained model, such as RoPE scaling.

You **can** choose a different topic if you found one in the papers.

A.5

Reverse-engineer each paper

For every design change, write down the thought process that led to it. Researchers usually change a block because they saw a problem. Find that problem. Use this chain for each change:

> **Problem seen → Idea → Change to the block → Result → Cost**
>
> Example from class. *Problem:* after attention, the token vector carries the context but loses its original meaning. *Idea:* keep the original and add the update to it. *Change:* the residual connection, $x + h$. *Result:* the token keeps both its meaning and its context. *Cost:* almost none, one addition.

Do this for at least five changes across the three models. The paper's own text often states the problem in the introduction or the ablation section.

A.6

How to write it

- Keep the paper academic and plain. Do not use marketing language.
- Use math only where it helps. Explain every equation in one sentence of plain English.
- Prefer figures to equations. Most readers follow a diagram of one block faster than its notation.
- You **can** use Overleaf. Search its template gallery for an arXiv preprint template and start from that.
- Cite every paper, video and post you used. Put them in the reading log.

A.7

Support

- If you need help with the structure of your paper, ask on Skool.
- If you show substantial progress, we can give you Tinker credits for a larger training run. Ask on Skool.
- If your paper is strong, we will help you publish it on arXiv. arXiv asks new authors for an endorsement, and we can give it for papers that meet the bar.

A.8

The structure of your white paper

Your paper **must** have these sections, in this order. The page counts are a guide.

#	SECTION	WHAT GOES IN IT	PAGES
1	Abstract	The question, what you compared, and your main finding. 150 to 200 words.	–
2	Introduction	Why these three models, and what a reader will learn.	0.5
3	Background	The path of one token through a decoder-only model, with one figure.	1
4	The abstraction lens	Your table from [A.3] and how to read it.	1
5	Block-by-block comparison	One subsection for each block. What each model chose, and why.	3 to 4
6	Deep dives	The topics from [A.4]: tokenizers, positions, heads, GQA and the extra topics you picked.	2 to 4
7	Design decisions, reverse-engineered	Your chains from [A.5].	1 to 2
8	Hyperparameters and trade-offs	Your completed table from [A.4] and what it tells you.	1
9	What generalizes	The ideas that all three models share, and how to use them to read the next paper.	0.5 to 1
10	Limitations	What you did not check, and where you are unsure.	0.5
11	References	Every source you cited.	–
A	Appendix: reading log	Each source, the date you read it, and one line on what you took from it.	–

PART B · EXPLAIN

The LoRA Video

Explain how LoRA changes a frozen model with two small matrices, A and B. Show it, step by step, with one worked example.

TEAMS You can do this part alone or in a team. To find teammates, post on Skool. In your submission, list what each person did.

B.1

Goal

In class we froze W_Q and added a small set of trainable weights next to it. This part asks you to explain that idea properly. Your viewer should understand why LoRA uses two matrices, what the rank r controls and what alpha (α) does.

Do not lead with the equation. Lead with the problem, and build up to the equation so that it looks like the obvious answer. Explain why each piece makes sense. Do not derive it.

B.2

Scope

- You **must** cover the frozen weight W, the matrices A and B, the rank r, alpha, how A and B start, and how the update is merged after training.
- You **should** use the example in [B.4], or one with the same idea.
- You **should not** cover QLoRA, DoRA, quantization or other LoRA variants. We cover them later.

B.3

The path: do not jump to the answer

Follow this order. Each scene should end with a question that the next scene answers. The times are a guide for a 15-minute video.

#	SCENE	WHAT THE VIEWER SHOULD UNDERSTAND	MIN
1	Where the weights are	W_Q, W_K, W_V, the FFN weights and the output head. Fine-tuning means changing some of them.	1.5
2	The problem	Full fine-tuning changes every weight. It needs a lot of memory, a full copy of the model per task, and it can damage what the model already knows.	1.5
3	Freeze, then add a change	Keep W fixed and learn only a change, ΔW. The new weight is $W + \Delta W$.	1.5
4	The change can be small	A new behavior usually needs only a few new directions, not a full new matrix. This is the low-rank idea.	2
5	Build ΔW from A and B	A compresses the input to r numbers. B expands them back to the output size. $B \times A$ has the same shape as W, so you can add them.	3
6	How A and B start	A starts with small random values. B starts at zero. So ΔW is zero, and the model starts exactly as it was.	1
7	What the rank does	Show $r = 1$, 4 and 8. A higher rank allows more directions of change and costs more weights.	1.5
8	What alpha does	The update is scaled by α/r. Alpha works like a volume knob for the change. Dividing by r keeps the volume steady when you change the rank.	1.5
9	Training and merging	Only A and B get updated. After training, add $\frac{\alpha}{r}BA$ into W. The model then runs at the same speed as before.	1
10	Back to your project	Read your Tinker config: the rank, alpha and target layers. Say why you chose them.	1

B.4

The worked example

Use the numbers from class so that the viewer can follow every shape.

The frozen weight. W_Q has 32 rows and 64 columns: $32 \times 64 = 2{,}048$ weights. We do not train these.

The two LoRA matrices, with rank $r = 4$. A has 4 rows and 64 columns: it reads the 64-number input and compresses it into 4 numbers. That is 256 weights. B has 32 rows and 4 columns: it turns those 4 numbers into a 32-number update. That is 128 weights.

The result. $B \times A$ is 32×64, the same shape as W_Q, so the two can be added. You train $256 + 128 = 384$ weights instead of 2,048, which is about 19%.

At real scale. For a $4{,}096 \times 4{,}096$ weight with $r = 8$, you train 65,536 weights instead of about 16.8 million. That is about 0.4%.

In class we described this as "four extra recipes" added to W_Q. Your video should show why one small matrix is not enough: its shape does not match W. Two matrices, one that compresses and one that expands, solve the shape problem and keep the count small.

B.5

Questions your video must answer

Answer each one with a reason that a non-specialist can accept.

- Why do we freeze W instead of training it?
- Why two small matrices, and not one?
- Why does B start at zero?
- What happens to quality and cost when you raise the rank?
- What does alpha do, and why is it divided by r?
- Why is there no extra cost at inference after you merge?
- Which layers should you adapt, and why might you choose attention, the FFN, or both?

B.6

The video

- The video **must** be 10 to 20 minutes long.
- You **should** make it with Manim (Community Edition). Animate the shapes of W, A and B, and show the multiplication and the addition.
- If you cannot use Manim, you **can** record a LaTeX (Beamer) presentation instead. It must still be visual and step by step.
- Record the voice-over yourself. Add captions.
- Put the animation source code in a public repository.
- Publish the video on YouTube or X. Clips from it count toward Part D.

B.7

The report

Write a report of 4 to 6 pages in LaTeX. It follows the same path as the video, in this order:

1. Introduction: what LoRA is for.
2. The problem with full fine-tuning.
3. The idea: freeze W and learn a small change.
4. A and B: shapes, how they start, and the worked example from [B.4].
5. Rank and alpha: what each one controls, with a small table of the trade-offs.
6. Training and merging.
7. Your choices for Part C, and why.
8. References.

Use stills from your video as figures.

PART C · BUILD

The SLM Project

Teach a small Qwen model one behavior with data that you make. Train it on Tinker. Prove that it learned the behavior.

TEAMS You can do this part alone or in a team. To find teammates, post on Skool. In your submission, list what each person did.

C.1

Goal

A small language model is useful when it does one job in a consistent way that a prompt cannot guarantee. In class, the job was to decide the next action in a sales conversation and to write the reply in the way the business wants. In this project you pick your own job, build the data, and fine-tune a small Qwen model with LoRA.

The project succeeds only if the fine-tuned model shows the behavior on test data that it never saw, and does it better than the base model. Most of the work is in the data. Training is a few lines of code.

C.2

Scope

- You **must** train with LoRA on Tinker, using a Qwen base model from the Tinker model list.
- You **must** build your own dataset. Synthetic data is allowed.
- You **should not** copy the class project. Use the class process on a new problem.
- You **should not** work on inference speed, agents or guardrails here. We cover them in later sessions.

C.3

Pick one behavior

Write the behavior in one sentence. Then write one paragraph on why a prompt is not enough. Good reasons are consistency over many cases, a policy that is specific to one organization, too many examples to fit in a prompt, and cost.

Define the input and the output. In class, the input was the campaign context and the message history, and the output was a decision (reply, hand off to a human, ask for clarification, follow up later) and a draft reply. A decision plus a draft is a good pattern because you can score the decision exactly.

EXAMPLES OF A GOOD SCOPE

A support assistant that hands high-value customers to a human. A code reviewer that applies one team's rules. A triage assistant that routes tickets with a fixed set of labels.

C.4

Build the dataset

- Collect or generate examples. Synthetic data is fine if it matches the situations the model will see.
- Store each example as one JSON line with the input, the decision and the draft.
- Remove personal data, such as names, emails and phone numbers.
- Split the data into training, validation and test sets. Do not look at the test set while you build the model.
- Start with at least 100 reviewed examples. Add more only if they are consistent.

For a portfolio project, you **can** add known flaws on purpose, for example contradictory labels, and show that your pipeline finds them. Document every flaw that you added.

C.5

Label and review

Use a model for the first pass of labels, and use people to review them. Write a review SOP: a short document that tells a reviewer how to decide each label. Two reviewers who follow the SOP should give the same label to the same example. For each example, record the label, the reviewer and whether the example was kept. Reject examples that do not teach the behavior.

C.6

Check that the pattern is learnable

Before you train, analyze the data.

- Look for contradictions: the same situation with different labels.
- Look at the label balance. A model trained on 95% "reply" learns to always reply.
- Read a random sample of 20 examples yourself.

Fix the data, then update the SOP so that the same problem does not come back. If the trained model later learns the wrong behavior, check the data first.

C.7

Set the baseline

Write your success criteria before you train. Then score two models on the test set:

- the Qwen base model, with your best prompt, and
- one frontier model, such as Claude or GPT, with the same prompt.

Score the decision by exact match. Score the draft with a written rubric, applied by a person or by a model acting as a judge. Keep the scoring method fixed for every model.

C.8

Choose the LoRA settings

Write your settings in a config file and explain each one in the README. Use what you learned in Part B.

SETTING	WHAT YOU MUST EXPLAIN
Base model	Why this size. A larger model costs more and may need less data.
Rank and alpha	Why this capacity for this amount of data. A high rank on a small dataset can memorize it.
Target layers	Attention, the FFN or both, and why.
Learning rate and epochs	How you chose them, and what the validation loss told you.
Cost	The estimated cost before the run, and the cap you set.

Do a dry run first. It checks the data format and the cost without training.

C.9

Train on Tinker

Load the training data, run the training loop (forward pass, loss, backward pass, optimizer step) and save the adapter weights. Log the training and validation loss for every epoch. Use the Tinker documentation as your main reference. If you need Tinker access or credits, ask on Skool.

C.10

Evaluate and report

Score the fine-tuned model on the test set with the same method as [C.7]. Put all three models in one scorecard.

- Show the score for each decision type, not only the total.
- Show at least five examples where the fine-tuned model failed, and say why you think it failed.
- Say what you would change in the data next.

You **can** also look at which parts of the input led to one output. This is a first step into interpretability.

C.11

The repository

Your public repository **must** have:

- A README with the behavior, the results and the steps to run the project.
- A commit history that shows the work in steps. One large commit at the end is not acceptable.
- The dataset (or a sample, if it is large), the review SOP and the config file.
- Two Mermaid diagrams. The first shows how one example moves through every file. The second shows the training script, step by step.
- The scorecard.
- No API keys. Keep them in an environment file that is not committed.

PART D · HABIT

Build in Public

Post 150 to 200 pieces of content on X and LinkedIn while you work on Parts A, B and C. There is no deadline.

Part D is individual. Each person posts from their own accounts, even if they work on A, B or C in a team.

D.1

Why we ask for this

When you explain something in public, you find out quickly whether you understand it. Your posts also become a record of your work that other people can check. Post while you work. Do not save everything for the end.

D.2

What counts as one piece

One piece is one post that teaches or shows one specific thing. It can be:

- a post or thread on X, or a post on LinkedIn,
- a figure or illustration that you made, with a short explanation,
- a short clip, for example from your Part B video,
- a walkthrough of a calculation, for example one token through attention in a spreadsheet,
- a milestone with evidence, for example your first fine-tuned model and its scorecard.

The same idea posted on X and on LinkedIn counts once.

D.3

Ideas for each part

WHILE YOU DO	YOU CAN POST
Part A	One block compared across Qwen, Llama and Mistral. How BPE splits a word. A figure of RoPE. Why GQA shares keys and values. One design decision, reverse-engineered. A hyperparameter and its trade-off.
Part B	Why LoRA needs two matrices. The shapes of A and B. What rank and alpha do. A clip from your Manim video. What you learned about Manim.
Part C	The behavior you picked and why a prompt is not enough. Your review SOP. A contradiction you found in the data. Your baseline score. A run that failed. The cost of a run. The final scorecard.
Any time	Something that confused you, and how you solved it. A mistake in an earlier post, and the correction.

D.4

How to write

- Write in your own words. Readers can tell when a post was written by AI without your thinking.
- Put one idea in each post.
- Show something real: a figure, a number, a code snippet or a screenshot.
- Say what was hard or what went wrong. These posts are often the most useful.
- Do not use hype words or generic openers.
- You **can** use AI tools to draw figures, fix grammar and test whether a draft reads as AI-written.
- You **can** use Manim for figures. We can share our Manim skill for coding tools. Ask on Skool.

D.5

Keep a log

Keep a simple sheet with one row for each piece: the date, the platform, the link, the part (A, B, C or other) and the topic. Share the link to your log with your final submission.